

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

DM546: COMPILER CONSTRUCTION

Full Interpreter

AUTHORS:

Lucas Bergholdt Hansen
lucha23@student.sdu.dk

Lasse Arpe Kristensen
lakri18@student.sdu.dk

Carl-Emil Dons Christensen
carlc23@student.sdu.dk

December 9, 2025



1 Preliminaries

If you used specific setups, not-well-known tools, or libraries briefly introduce them. Provide references to those artifacts. Note: you do not need to repeat what we discussed in the lectures and exercises.

2 Technical Description

Here you explain the technical work you have carried out. You should present the overall architecture of your interpreter. Try to explain the development choices you made taking into account important properties, such as reusability and performance.

2.1 Architecture

Please provide at least **one diagram** of the entire architecture and describe it. How do the individual elements of the interpreter (i.e., scanner, parser, etc.) work together to execute a given program? Have you applied specific design patterns?

2.2 Implementation

If you have implemented specific algorithms or used functionality not covered in the lecture, please explain it here. Which kind of design decisions did you take? Answer the following questions in the respective sections.

- How do you recognize tokens in the Scanner?
- How does the Parser work?
- How do you perform the semantic analyses?
- How do you handle the casting of types?
- How do you handle functions?
- What tasks are performed by the interpreter?
- How and when do you report errors?

2.2.1 Syntactic Analysis

2.2.2 Semantic Analysis

After having making sure that the program is syntactically correct, we want to confirm that our context-sensitive grammar is also semantically correct, i.e. perform context-sensitive

checks and interpret the meaning of the program at its' syntactic level. VVPL is a statically typed language, which usually means that we perform context-sensitive checks at compile-time. VVPL is however an interpreted language, meaning that we do not compile the code directly to native machine code, but let the interpreter execute the code. This means that in our case, "statically-typed" means that we perform the context-sensitive checks before interpreting the program (runtime evaluation).

Context-sensitive checks are performed by constructing symbol tables upon entering a new scope. VVPL practises lexical (or static) scoping by using curly brackets to indicate a new scope. An example of constructing a new symbol table to represent the scope of the current block can be seen in the visitBlockStmt:

```
1 public Void visitBlockStmt(BlockStmt blockStmt) {  
2     SymbolTable oldTable = currentEnvironment;  
3     currentEnvironment = new SymbolTable(currentEnvironment);  
4     for (int i = 0; i < num_statements; i++) {  
5         analyse(blockStmt.stmts.get(i));  
6     }  
7     currentEnvironment = oldTable;  
8     return null;  
9 }
```

Here we construct a new symbol table which represents the scope of the current block. The new symboltable has the old symbol table as an attribute such that it is possible to access variables from outer scopes, where as variables in inner scopes will be unreachable.

Scope Analysis

Roughly we handle three things in the scopeanalyzer, being variables, functions declarations and function calls. We will go through them step-by-step

- **Variables** The following scopeing rules fall in this category (omitting the variables within functions for now):

- Variable can only be declared once by a variable declaration
- Variables can not be shadowed (i.e. defined in an inner scope to shadow the object in the outer scope)
- Variable declarations needs to be initialized with a value
- Variable must be visible in current scope before usage

scopeanalyzer.java enforces the first two bullet points in the logic of SymbolTable.java. Upon attempting to define, we check if the variable exists in the current or any outer tables to prevent shadowing.

```
1 public void define(String symbol, Token token) throws ←  
    SymbolTableException {  
2     if (contains(symbol)) {
```

```

3         throw new SymbolTableException();
4     }
5     symbols.put(symbol, token);
6 }
7     public boolean contains(String symbol) {
8         if (symbols.containsKey(symbol)) {
9             return true;
10        }
11
12        if (outer != null) {
13            return outer.contains(symbol);
14        }
15        return false;
16    }

```

We that declarations needs to be initialized by a value in visitVarDecl:

```

1 if (varDecl.expr != null) {
2     analyse(varDecl.expr);
3 }
4 else {
5     VVPLController.error(varDecl.id.line, ErrorTypeStrings.SCOPE_ERROR, "↵
6     declared variable must be initialized with a value.");
7 }

```

In visitIdentifier we enforce the fourth bullet point by checking if the symbol table contains the identifier:

```

1 public Void visitIdentifierExpr(Identifier identifier) {
2     if (!currentEnvironment.contains(identifier.id.lexeme)) {
3         VVPLController.error(identifier.id.line, ErrorTypeStrings.↵
4         SCOPE_ERROR, "variable [insert name here] does not exist in scope or ↵
5         any parent scopes.");
6     }
7 }

```

- **Function Declarations** The following scoping rules fall in this category:

- Variables inside the function declaration cannot access any outer scopes. These must be **shadowed**
- cannot be defined in inner scopes, only in the global scope
- Unique function name (i.e. re-declarations are not possible)
- Return statements can only occur in the context of a function
- No statements can follow a return statement
- Out-of-order declarations and calls allowed, i.e. a function call can precede its' declaration

In `visitFunctionStmt`, we enforce the first bulletpoint by constructing a new symbol table with no outer symboltable for functions. This way they cannot interact with any values defined outside of this block. The second bulletpoint is enforced by checking whether the current environment is the original defined environment for the program. This environment is made unique by adding a separate constructor for a `symbolTable` marking it as the global environment. One could think we could check whether our global environment is global by seeing if there are no outer symbol tables linked to the current symboltable, but due to our solution for encapsulating functions in their own scope, this is not possible.

We have constructed a separate symbol table class to contain functions - this of course has the consequence that a variable and a function can share the same name, but it beautifies the implementation of our scopeanalyzer significantly. This new table maps a symbol not only to their token, but to their respective `FunctionStmt` AST node. This way

One could think this could be checked by seeing whether a symbol table has an outer table, but we will see later why this is not possible. We do this by having an attribute

- **Function Calls** The following scopeing rules fall in this category:

- Function arguments must be within scope
- Number of function arguments matches number of function parameters
- Only declared and known functions can be called

lidt om lexical scoping... VVPL We utilize lexical, or static, scoping by using By using curly brackets to indicate a scope, we utilize lexical

The Scope Analysis is performed by the class `ScopeAnalyzer.java`

Type Analysis

2.2.3 Tree-Walk Interpretation

2.2.4 Monitoring and Logging

An essential part of making a programming language usable includes having a sensible error handling. Clear error messages help users understand what went wrong in their program and how to fix it. To facilitate this, we perform error handling in all parts of the interpreter.

At the center of the error handling is the `VVPLController` class which functions as a centralized hub for reporting errors. This design decouples the error *detection* from the error *reporting*. Each component such as the scanner or parser is responsible for identifying

errors and notifying the `VVPLController` class which in turn is responsible for handling how these errors are stored and reported to the user.

This interplay between the classes is facilitated by the `error` method in the controller.

```
1 public static void error(int line, String errorType, String message) {  
2     hadError = true;  
3     errorMessages.add(new ErrorMessage(line, errorType, message));  
4 }
```

This method takes in the line where the error occurred, the type of error and the error message we are reporting to the user. The error type is one of the different errors from the `ErrorTypeStrings` class, which allows us to categorize the errors. The method firstly sets the `hadError` flag, which signals that the current stage of compilation has failed. After each stage of the interpreter pipeline, the controller checks this flag. If an error has been registered the pipeline stops. Halting the interpreter as soon as a single stage fails is part of a fail fast strategy that prevents later stages from operating on invalid data which would cause a cascade of confusing and possibly misleading errors. While the pipeline stops between stages, each individual stage is designed to detect as many errors as possible before terminating, providing the user with a comprehensive report of their errors in that stage. This gives the user a more complete overview of issues which decreases time spent debugging by reducing the number of compiles needed to find and fix all errors in the program.

After setting the `hadError` flag the error is added to the `errorMessages` list which stores all the errors. This is a list of `ErrorMessage` objects:

```
1 private static List<ErrorMessage> errorMessages = new ArrayList<>();
```

The `ErrorMessage` class stores errors as structured data rather than just plain strings. It stores the line number, error type and message in separate fields which allows for easy post-processing. Specifically this allows for sorting the errors by line number such that the user receives a logical and actionable list of issues in the order they appear in the code. This is achieved by having `ErrorMessage` implement the `Comparable` interface allowing the controller to sort the list with the `sort` method from the `Collections` class. The class also overrides the `toString` method such that the error messages can easily be reported to the user in the format: `<error-type>, line <lineNo> <message>`.

2.2.5 Lexical error handling in the Scanner

The `Scanner` is the first component to utilize this centralized error handling. When it encounters a character or character sequence that does not conform with the language, it calls the controller's `error` method. A common example is finding a character that is not part of the language such as '\$'. This is handled in the default branch of the `scanToken` method:

```
1 private void scanToken() {
```

```
2  char c = advance();
3  switch (c) {
4      // ... cases for valid characters
5      default:
6          if (isDigit(c)) {
7              number();
8          } else if (isAlpha(c)) {
9              identifier();
10         } else {
11             VVPLController.error(line, ErrorTypeStrings.SCAN_ERROR, "↵
Unexpected character.");
12         }
13     }
14 }
```

A deliberate design choice here is that the scanner does not halt after finding an error. Instead it simply reports the error and proceeds to the next character, attempting to tokenize the rest of the file. This allows the scanner to report all lexical errors in a single pass, rather than informing the user about the errors one by one.

One area, however, requires extra care: handling string literals. When encountering a string...

2.2.6 Syntactic error recovery in the Parser

While the scanner can often recover from errors by simply advancing, handling errors in the parser is more involved. The parser should be robust in the face of syntax errors such that it does not crash or hang. Invalid code is valid input for a parser since one of its key responsibilities is to teach the user what valid syntax looks like. A single syntax error like a missing parenthesis may confuse the parser and cause it to report a long cascade of pseudo-errors that don't indicate another real problem in the code. We should strive to report only errors that are caused by the user's invalid syntax and not by the parser's own confusion after encountering an error.

To support both reporting as many distinct errors as possible while also filtering out those who are simply a side-effect of another error we have implemented the more sophisticated error recovery strategy known as *panic mode*.

2.2.7 Panic mode and synchronization

When the parser detects an error it enters panic mode. At this point the parser knows that at minimum one token is unexpected in the current state of applying a production rule from the grammar and that the parser's internal state and the remaining token stream may no longer be aligned. Before parsing can continue, the parser must realign its state with the upcoming tokens such that the next token matches the expected token in the rule

being parsed.

This is accomplished through synchronization. The idea is to choose a specific grammar rule as synchronization point that is safe to restart from and to discard tokens until a token that is valid at this synchronization point is encountered. Our chosen synchronization point follows the common decision of choosing to synchronize between statements.

The implications of this approach is that any real syntax errors hiding inside the discarded tokens will not be reported. However, this is an acceptable tradeoff for avoiding falsely reporting cascaded errors in the discarded tokens that are simply a side effect of the initial error.

2.2.8 Synchronizing via exceptions

In a recursive descent parser like ours the parser's state is not explicitly stored but instead tracked with the call stack. So to reset the parser's state we need to clear out the call frames on the stack. We achieve this by using Java exceptions. Encountering a syntax error triggers a `ParseError` exception, which unwinds the stack until it is caught in the `parse` loop between statements.

The exception is triggered through the parser's `error` method, which informs the controller about the error and produces the exception:

```
1 private ParseError error(Token token, String message) {  
2     VVPLController.error(token.line, ErrorTypeStrings.PARSE_ERROR, message);  
3     return new ParseError();  
4 }
```

The exception itself is just a simple class:

```
1 private static class ParseError extends RuntimeException {}
```

When a `ParseError` is thrown, the top-level loop in `parse` catches it and invokes the `synchronise` method. This method discards tokens until reaching a token that can begin a new statement. The `synchronise` method can be seen in its full glory below. It mainly consists of just a simple switch statement in a loop. We are also returning if we have just discarded a semicolon as this is a good indication, that a new statement is likely to start.

```
1 private void synchronise() {  
2     advance(); // Discard the problematic token that triggered the error  
3  
4     while (!isAtEnd()) {  
5         if (previous().type == SEMICOLON) return;  
6  
7         switch (peek().type) {  
8             case FUNCTION:  
9             case VAR:  
10            case IF:
```



```
11         case WHILE:
12         case PRINT:
13         case LEFT_BRACE:
14         case RETURN:
15             return;
16
17         default:
18             advance(); // Discard token and continue
19             break;
20     }
21 }
22 }
```

The `error` method is also integrated into `consume`, supporting reusability and enabling consistent syntax checking throughout the parser:

```
1 private Token consume(TokenType type, String message) {
2     if (check(type)) {
3         return advance();
4     } else {
5         throw error(peek(), message);
6     }
7 }
```

This parsing strategy ensures that the parser reports only genuine syntax errors, recovers cleanly and avoids polluting the error output with misleading cascading errors.

2.3 Testing & Evaluation

How did you test your interpreter? Which coverage criterion did you apply?

Describe the test setup and the test inputs with their expected outputs. If you have documented this in the source code a link and submission of the generated JavaDoc is also fine.

If the provided tests have been changed or do not work, please explain here what tests fail, why and what it would require to fix them.

3 Teamwork Organization

Describe how you organized the work together in the team? How did you communicate, implement and distribute tasks?

Additionally, reflect on your individual learning outcomes (particularly if you worked alone).

4 Discussion

In this section you review possible alternative solutions to the same challenge you have tried to address in your project, or alternative methodologies that you may have followed (e.g., choice of other technologies for implementing the project). Provide a discussion on the implications of your choices in the design of your work and the technologies/techniques that you have used.

Discuss what are the shortcomings/limitations of your project, possibly explaining how they could be solved or mitigated.